

ISM4641: Python for Business Analytics

Lecture 3: Problem Set Solutions

Dr. Tim Smith

February 15, 2025

Table of contents

Part I: Tuples (10 Questions)	2
1. Creating Your First Tuple	2
2. Indexing and Slicing	2
3. Immutable Operations	3
4. Unpacking Tuples	4
5. Tuple with Mixed Data Types	4
6. Counting and Index Methods	5
7. Nested Tuples	6
8. Tuple Packing	7
9. Swapping Values	7
10. Data Records & Tuples	8
Part II: Sets (10 Questions)	10
1. Set Creation	10
2. Removing Duplicates	11
3. Membership Testing	11
4. Set Operations (Union, Intersection, Difference)	12
5. Symmetric Difference	13
6. Add, Remove, Discard	14
7. Pop Operation	14
8. Set of Sets?	15
9. Frozenset (advanced)	16
10. Practical Use Case: Course Enrollment	17
Final Remarks	18

Part I: Tuples (10 Questions)

1. Creating Your First Tuple

Question:

> 1. **Creating Your First Tuple**

> - **Task:** Create a tuple named `employee_info` with the values "E123", "John Doe", and 35.

> - **Question:** How would you print each element in `employee_info` individually, referencing indexes?

Solution

```
1  # Creating the tuple
2  employee_info = ("E123", "John Doe", 35)
3
4  # Printing each element using indexes
5  print("Employee ID:", employee_info[0])
6  print("Employee Name:", employee_info[1])
7  print("Employee Age:", employee_info[2])
```

Explanation:

1. Tuples are created using parentheses ().
 2. Each piece of information—"E123", "John Doe", and 35—forms an element in the tuple.
 3. We use zero-based indexing: `employee_info[0]` is "E123", `employee_info[1]` is "John Doe", and `employee_info[2]` is 35.
 4. Tuples are immutable, meaning we cannot change these values later, but accessing them by index is perfectly valid.
-

2. Indexing and Slicing

Question:

> 2. **Indexing and Slicing**

> - **Task:** Given the tuple `days = ("Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun")`, use slicing to create a smaller tuple of just the weekdays.

> - **Question:** Why might you prefer a tuple over a list for something like day names?

Solution

```
1 days = ("Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun")
2
3 # Slicing out only the weekdays (Mon-Fri), indexes 0 through 4
4 weekdays = days[0:5]
5
6 print("All days:", days)
7 print("Weekdays:", weekdays)
```

Explanation:

1. Slicing syntax `days[start:stop]` allows extracting a subset of a tuple. Here, `0:5` grabs elements at indices 0, 1, 2, 3, 4.
 2. We might prefer a tuple over a list for day names because day names are constant and unlikely to change. Tuples' immutability helps signal that these data are fixed and should remain intact.
-

3. Immutable Operations

Question:

> 3. Immutable Operations

> - **Task:** Suppose you attempt `days[0] = "Monday"` on the `days` tuple.

> - **Question:** What error do you get, and why does this illustrate tuple immutability?

Solution

```
1 days = ("Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun")
2
3 # Attempting to modify the tuple
4 try:
5     days[0] = "Monday"
6 except TypeError as e:
7     print("Error encountered:", e)
```

Explanation:

1. Running the code above raises a **TypeError** with a message like `'tuple' object does`

not support item assignment.

2. This occurs because tuples are **immutable**—they cannot be changed once created. Any attempt to assign a new value to an existing index will fail.

4. Unpacking Tuples

Question:

> 4. **Unpacking Tuples**

> - **Task:** You have a tuple `dimensions = (1920, 1080)`. Use tuple unpacking to assign the width and height to separate variables named `width` and `height`.

> - **Question:** Why might unpacking be more convenient than accessing each index separately?

Solution

```
1 dimensions = (1920, 1080)
2
3 # Tuple unpacking
4 width, height = dimensions
5
6 print("Width:", width)
7 print("Height:", height)
```

Explanation:

1. **Tuple unpacking** lets you assign multiple variables in a single line, which can be more readable and concise than writing `width = dimensions[0]` and `height = dimensions[1]`.
 2. It also helps avoid mistakes by matching variables directly to the positions of the tuple's contents.
-

5. Tuple with Mixed Data Types

Question:

> 5. **Tuple with Mixed Data Types**

> - **Task:** Create a tuple named `product_record` containing:

> 1. A string: "P-XYZ"
> 2. A float: 29.99
> 3. An integer: 100
> - **Question:** Print each element. What does this demonstrate about Python's flexibility with data types in tuples?

Solution

```
1 product_record = ("P-XYZ", 29.99, 100)
2
3 print("Product ID:", product_record[0])
4 print("Price:", product_record[1])
5 print("Quantity:", product_record[2])
```

Explanation:

1. Python allows tuples (and other sequences) to store elements of *different types*.
 2. Here we have a string ("P-XYZ"), a float (29.99), and an integer (100), all in the same tuple.
 3. This demonstrates Python's flexibility, as there is no strict requirement for uniform data types within a tuple.
-

6. Counting and Index Methods

Question:

> 6. Counting and Index Methods
> - **Task:** Given the tuple `nums = (4, 2, 4, 1, 4)`, find:
> 1. How many times the integer 4 appears.
> 2. The index of the integer 1.
> - **Question:** Which tuple methods would you use for these tasks?

Solution

```
1 nums = (4, 2, 4, 1, 4)
2
3 # Count how many times '4' appears
4 count_4 = nums.count(4)
```

```

5
6 # Find the index of '1'
7 index_1 = nums.index(1)
8
9 print("Number of times '4' appears:", count_4)
10 print("Index of '1':", index_1)

```

Explanation:

1. `.count()` returns the number of occurrences of a given value within a tuple.
 2. `.index()` returns the first position at which a given value appears. If the value is not in the tuple, it raises a `ValueError`.
 3. This question highlights two common tuple methods used for quick checks and lookups.
-

7. Nested Tuples

Question:

> 7. Nested Tuples

> - **Task:** Create a tuple named `employee_db` that contains two smaller tuples, each representing an employee's (ID, Name, Age).

> - **Question:** How can you print Bob's age using nested indexing?

Solution

```

1 employee_db = (
2     ("E101", "Alice", 30),
3     ("E102", "Bob", 28)
4 )
5
6 # Accessing Bob's age using nested indexing
7 bob_age = employee_db[1][2]
8
9 print("Bob's Age:", bob_age)

```

Explanation:

1. Each element in the outer tuple is itself another tuple with three elements.
2. To get Bob's age, we look at index 1 of `employee_db` (the second employee), then index 2 of that inner tuple (the Age field).
3. This is an example of **nested data** and how indexing extends to multiple levels.

8. Tuple Packing

Question:

> 8. **Tuple Packing**

> - **Task:** Write a short script that creates four separate variables (`city`, `country`, `population`, `continent`) and then uses **tuple packing** to combine them into a single tuple `location_info`.

> - **Question:** How is packing different from defining a tuple directly in parentheses?

Solution

```
1  # Four separate variables
2  city = "Paris"
3  country = "France"
4  population = 2148327
5  continent = "Europe"
6
7  # Tuple packing
8  location_info = city, country, population, continent
9
10 print("Location Info (packed tuple):", location_info)
```

Explanation:

1. **Tuple packing** happens implicitly when you separate variables or values by commas without enclosing them in parentheses. Python automatically forms a tuple from these items.
2. It is slightly different from writing `(city, country, population, continent)` explicitly, but both achieve the same effect of creating a tuple.
3. The main distinction is that the tuple is formed “on the fly” when using comma separation in an assignment statement.

9. Swapping Values

Question:

> 9. **Swapping Values**

> - **Task:** Suppose you have `a = 10` and `b = 20`. Use a tuple assignment approach to swap these two values in one line of code.

> - **Question:** Why is this “swapping trick” often demonstrated when teaching tuples?

Solution

```
1 a = 10
2 b = 20
3
4 # Swapping using tuple assignment
5 a, b = b, a
6
7 print("After swapping:")
8 print("a =", a)
9 print("b =", b)
```

Explanation:

1. Python's tuple assignment lets you swap values in a single statement without needing a temporary variable.
 2. This technique is commonly demonstrated to show how Python's multiple-assignment features (which rely on tuples under the hood) can simplify tasks like swapping.
-

10. Data Records & Tuples

Question:

> 10. **Data Records & Tuples**

> - **Task:** Imagine you have a point-of-sale system that logs transactions in the format (transaction_id, amount, timestamp). Create three sample tuples representing different transactions, and put them in a list called `transactions_list`.

> - **Question:** What advantages might tuples offer in scenarios where data integrity is important?

Solution

```
1 # Creating three sample transaction tuples
2 t1 = ("T001", 100.50, "2025-01-10 10:00:00")
3 t2 = ("T002", 250.00, "2025-01-10 10:05:00")
4 t3 = ("T003", 99.99, "2025-01-10 10:07:00")
5
6 # Putting them in a list
```



```
7 transactions_list = [t1, t2, t3]
8
9 # Printing all transactions
10 for transaction in transactions_list:
11     print(transaction)
```

Explanation:

1. Each transaction is stored as a **tuple** with (transaction_id, amount, timestamp).
 2. By using tuples, we ensure that the core structure (three elements in a specific order) does not change. This can help preserve data integrity in systems that require consistent records.
 3. Storing them in a list allows iteration, modification of the overall collection (like adding or removing entire transaction tuples), but not altering the transaction records themselves.
-

Part II: Sets (10 Questions)

1. Set Creation

Question:

> 1. Set Creation

> - **Task:** Create an empty set called `favorite_fruits`. Then add "apple", "banana", and "mango" to it.

> - **Question:** How do you verify that a set is empty before adding new elements?

Solution

```
1  # Creating an empty set
2  favorite_fruits = set()
3
4  # Verifying it's empty
5  if len(favorite_fruits) == 0:
6      print("Set is empty.")
7
8  # Adding elements
9  favorite_fruits.add("apple")
10 favorite_fruits.add("banana")
11 favorite_fruits.add("mango")
12
13 print("Favorite fruits:", favorite_fruits)
```

Explanation:

1. An empty set can be created with `set()`. Using `{}` alone creates an empty dictionary, not a set.
 2. We can check if it's empty by verifying `len(favorite_fruits) == 0`.
 3. Sets store unique elements, so any duplicates added subsequently would be ignored (though not in this exact example).
-

2. Removing Duplicates

Question:

> 2. **Removing Duplicates**

> - **Task:** You have a list `numbers = [1, 2, 2, 3, 3, 3]`. Convert this list to a set and store it in `unique_numbers`.

> - **Question:** Why does converting `numbers` to a set automatically remove duplicates?

Solution

```
1 numbers = [1, 2, 2, 3, 3, 3]
2
3 # Converting list to set
4 unique_numbers = set(numbers)
5
6 print("Original list:", numbers)
7 print("Set with unique elements:", unique_numbers)
```

Explanation:

1. By definition, a set in Python only contains **unique** items.
 2. When converting a list to a set, any duplicates are discarded because sets cannot have repeated elements.
 3. That is why the resulting `unique_numbers` set contains only 1, 2, and 3.
-

3. Membership Testing

Question:

> 3. **Membership Testing**

> - **Task:** Given `favorite_fruits`, check whether "banana" is in the set and whether "kiwi" is in the set. Print the results.

> - **Question:** What is the syntax and computational benefit of using `in` on a set rather than on a list?

Solution

```

1 favorite_fruits = {"apple", "banana", "mango"}
2
3 print("Is 'banana' in favorite_fruits?", "banana" in favorite_fruits)
4 print("Is 'kiwi' in favorite_fruits?", "kiwi" in favorite_fruits)

```

Explanation:

1. We use the `in` operator to test membership: `"banana" in favorite_fruits` returns `True`, while `"kiwi" in favorite_fruits` returns `False`.
2. **Sets** are implemented as hash tables in Python, so membership tests (`in`) are generally $O(1)$ on average, meaning they are very fast. In contrast, membership testing on lists is $O(n)$, which can be slower when `n` is large.

4. Set Operations (Union, Intersection, Difference)

Question:

> 4. **Set Operations (Union, Intersection, Difference)**

> - **Task:** Given `set_a = {1, 2, 3}` and `set_b = {3, 4, 5}`, find:

> 1. The union of `set_a` and `set_b`.

> 2. The intersection of `set_a` and `set_b`.

> 3. The elements in `set_a` but not in `set_b`.

> - **Question:** What specific operators (or methods) would you use to perform each operation?

Solution

```

1 set_a = {1, 2, 3}
2 set_b = {3, 4, 5}
3
4 # Union
5 union_set = set_a | set_b
6 # Intersection
7 intersection_set = set_a & set_b
8 # Difference
9 difference_set = set_a - set_b
10
11 print("Union:", union_set)

```

```
12 print("Intersection:", intersection_set)
13 print("Difference:", difference_set)
```

Explanation:

1. **Union** (`|` or `.union()`) combines all unique elements from both sets.
2. **Intersection** (`&` or `.intersection()`) takes only elements found in both sets.
3. **Difference** (`-` or `.difference()`) takes elements in the first set that are not in the second.
4. These operators are shorthand for the corresponding methods (e.g., `set_a.union(set_b)`).

5. Symmetric Difference

Question:

> 5. Symmetric Difference

> - **Task:** With the same sets (`set_a` and `set_b`), find their symmetric difference.

> - **Question:** How does symmetric difference differ from regular difference or union?

Solution

```
1 set_a = {1, 2, 3}
2 set_b = {3, 4, 5}
3
4 # Symmetric difference
5 sym_diff = set_a ^ set_b
6
7 print("Symmetric Difference:", sym_diff)
```

Explanation:

1. **Symmetric difference** (`^`) is the set of elements that are in either `set_a` or `set_b`, **but not both**.
2. It is different from the regular **difference** in that difference is one-directional (`set_a - set_b`). Meanwhile, **union** includes everything from both sets (including overlaps). The symmetric difference excludes the overlap (3 in this case).

6. Add, Remove, Discard

Question:

> 6. Add, Remove, Discard

> - **Task:** Start with `store_items = {"laptop", "mouse", "keyboard"}`. Then do the following:

> 1. Add "monitor".

> 2. Remove "mouse".

> 3. Discard "printer" (which doesn't exist in the set).

> - **Question:** How do `remove` and `discard` differ if the element does not exist in the set?

Solution

```
1 store_items = {"laptop", "mouse", "keyboard"}
2
3 # 1. Add "monitor"
4 store_items.add("monitor")
5
6 # 2. Remove "mouse"
7 store_items.remove("mouse")
8
9 # 3. Discard "printer" (doesn't exist)
10 store_items.discard("printer")
11
12 print("Updated store_items:", store_items)
```

Explanation:

1. `store_items.add("monitor")` inserts a new element into the set if it isn't already there.

2. `store_items.remove("mouse")` removes "mouse", but if "mouse" hadn't existed, it would raise a `KeyError`.

3. `store_items.discard("printer")` attempts to remove "printer", but does **not** raise an error if "printer" is missing. That is the key difference between `remove` and `discard`.

7. Pop Operation

Question:

> 7. Pop Operation

> - **Task:** Use `pop()` on `store_items` and store the popped value in a variable named `removed_item`. Print it out, and then print the set.
> - **Question:** Why can't you be certain which item gets removed by `pop()`?

Solution

```
1 store_items = {"laptop", "keyboard", "monitor"}
2
3 removed_item = store_items.pop()
4 print("Removed Item:", removed_item)
5 print("store_items after pop():", store_items)
```

Explanation:

1. The `pop()` method on sets removes **an arbitrary element**, not necessarily the “first” because sets do not preserve order.
 2. You cannot predict which item will be removed by `pop()`, which is why the operation is described as removing an arbitrary element.
-

8. Set of Sets?

Question:

> 8. Set of Sets?

> - **Task:** Try creating a set that contains another set, for example: `my_set = {{1, 2}, 3, 4}`.

> - **Question:** What happens when you run this code, and why?

Solution

Impossible Code Attempt:

```
1 # Trying to create a set of sets
2 try:
3     my_set = {{1, 2}, 3, 4}
4 except TypeError as e:
5     print("Error encountered:", e)
```

Explanation:

1. You'll get a **TypeError**: 'unhashable type: 'set'.
 2. In Python, **set elements must be immutable** (hashable). A standard **set** (like {1, 2}) is mutable, so it cannot be placed inside another set.
 3. If you want a set inside another set-like structure, you'd have to use **frozenset**({1, 2}) which is immutable.
-

9. Frozenset (advanced)

Question:

> 9. **Frozenset**

> - **Task:** Use a **frozenset** to represent an immutable set of days off: **frozenset_days_off = frozenset(["Saturday", "Sunday"])**. Then try to add or remove an element.

> - **Question:** What happens, and how does **frozenset** differ from a standard set in terms of immutability?

Solution

```
1 frozenset_days_off = frozenset(["Saturday", "Sunday"])
2
3 print("frozenset_days_off:", frozenset_days_off)
4
5 # Attempting to add or remove an element
6 try:
7     frozenset_days_off.add("Friday")
8 except AttributeError as e:
9     print("Error adding element:", e)
10
11 try:
12     frozenset_days_off.remove("Sunday")
13 except AttributeError as e:
14     print("Error removing element:", e)
```

Explanation:

1. A **frozenset** is an **immutable** version of a set.
2. Because it is immutable, operations like **.add()** or **.remove()** do not exist and raise an **AttributeError**.

3. While a normal set allows adding and removing elements, a `frozenset` is fixed once created, similar to how a tuple is an immutable version of a list.

10. Practical Use Case: Course Enrollment

Question:

> 10. Practical Use Case: Course Enrollment

> - **Task:** Assume `course_python = {"Alice", "Bob", "Charlie"}` and `course_data_science = {"Bob", "Diana", "Charlie"}`. You want to find:

> 1. Students taking both courses.

> 2. Students only taking the Python course.

> 3. Students in at least one of the two courses.

> - **Question:** Based on these sets, how would you approach each requirement, and what does each result signify in a real-world scenario?

Solution

```
1 course_python = {"Alice", "Bob", "Charlie"}
2 course_data_science = {"Bob", "Diana", "Charlie"}
3
4 # 1. Students taking both courses
5 both_courses = course_python & course_data_science
6
7 # 2. Students only in Python (Python minus Data Science)
8 only_python = course_python - course_data_science
9
10 # 3. Students in at least one course (Union)
11 either_course = course_python | course_data_science
12
13 print("Taking both courses:", both_courses)
14 print("Only taking Python:", only_python)
15 print("Taking at least one of the two courses:", either_course)
```

Explanation:

1. We use **intersection** (&) to find those enrolled in both sets: {"Bob", "Charlie"}.
2. The **difference** (-) gives who is in Python but not in Data Science: {"Alice"}.
3. The **union** (|) finds every student in either or both courses: {"Alice", "Bob", "Charlie", "Diana"}.

4. In a real-world scenario, these sets let us quickly see overlaps, identify unique participants, or plan resources (e.g., if a student is in both courses, we might tailor scheduling or check for workload conflicts).

Final Remarks

These solutions demonstrate the essential operations and behaviors of **tuples** and **sets** in Python. Key takeaways include:

- **Tuples:**
 - Ordered, immutable sequences.
 - Ideal for data that shouldn't change.
 - Support quick multiple-assignment and can serve as dictionary keys.
- **Sets:**
 - Unordered, unique collections.
 - Excellent for membership tests and removing duplicates.
 - Useful for set operations like union, intersection, and difference.